

ENCM 515: ASPA – Lab 2

Group #	23	Date: 27-02-2025
Student name(s)	Lee Evans	
	Sudarshan Naicker	
Instructor/TA		

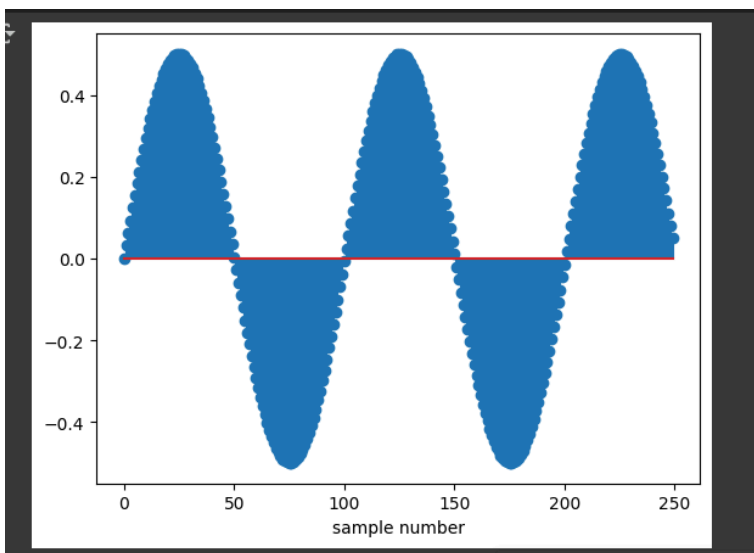
To submit:

- Add your lab sheet (as PDF if possible) and any other added/modified .c or .h files to a zip archive and upload to D2L.

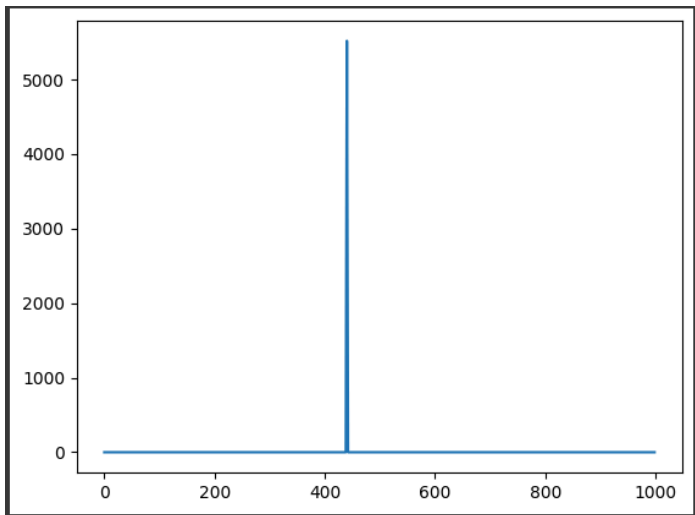
PART ONE

Q1> Screenshots of the first 250 filtered samples and the spectrum (from floating point FIR)

Data before filtering



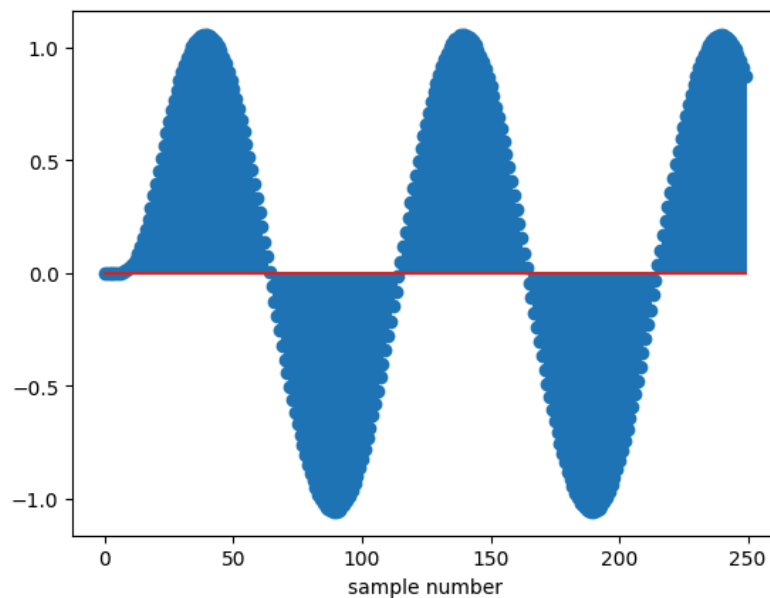
FFT spectrum diagram before filtering



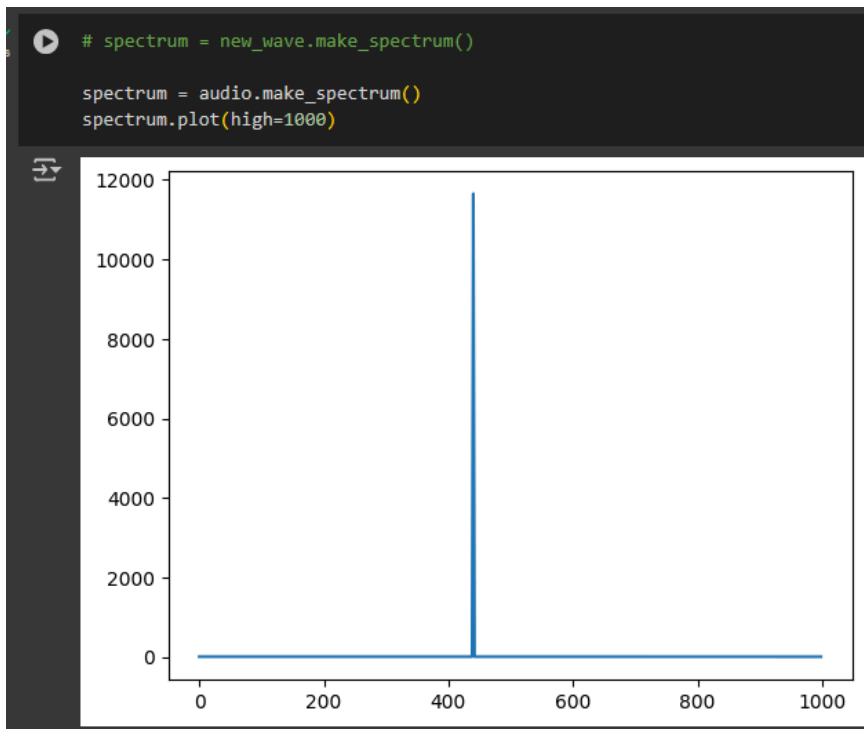
Data address after filtering

newdata	float [22050]	0x200002bc <newdata>
> [0...9999]	float [10000]	0x200002bc <newdata>
> [10000...19999]	float [10000]	0x20009efc <newdata+400...
> [20000...22049]	float [2050]	0x20013b3c <newdata+80...
+ Add new expression		

Data after filtering. We see only an amplitude scale plus a small amount of manipulation from the FIR.



FFT spectrum diagram of the filtered data. This makes sense since our FIR filter shouldn't have changed our pure tone signal since all frequency elements are less than the cutoff frequency of the filter.



How much time is taken to filter the signal? Is this filtering going to be “good enough” for our sample rate?

Index	Type	Data	Cycles	Time(s)	Extra info
0	Sync	96	1960932 ?	?	No timestamp received ...
1	Sync	48	1960932 ?	?	No timestamp received ...
2	ITM Port 31	1	1960932	20.426375 ms	
3	Sync	48	5960930	62.093021 ms	
4	Sync	48	21960922	228.759604 ms	
5	Sync	48	37960914	395.426188 ms	
6	Sync	48	55960905	582.926094 ms	
7	ITM Port 31	2	62850305	654.690677 ms	
8	Sync	48	0	0.000000 s	

```
161 FloatFilterInit();
162
163 ITM_Port32(31) = 1;
164 FloatFilterStart();
165 ITM_Port32(31) = 2;
```

we can see that the filter took $[654.69\text{ms} - 20.42\text{ms}] = 634.27\text{ms}$ to filter the 22050 samples. This equates to $2.88\text{e-}5\text{s}$ per sample or $28.8\mu\text{s}$.

If we wanted to filter in real time, then the speed of our filtering must be at least the speed of our sampling, but should be faster in case the system needs to execute interrupts or other asynchronous tasks.

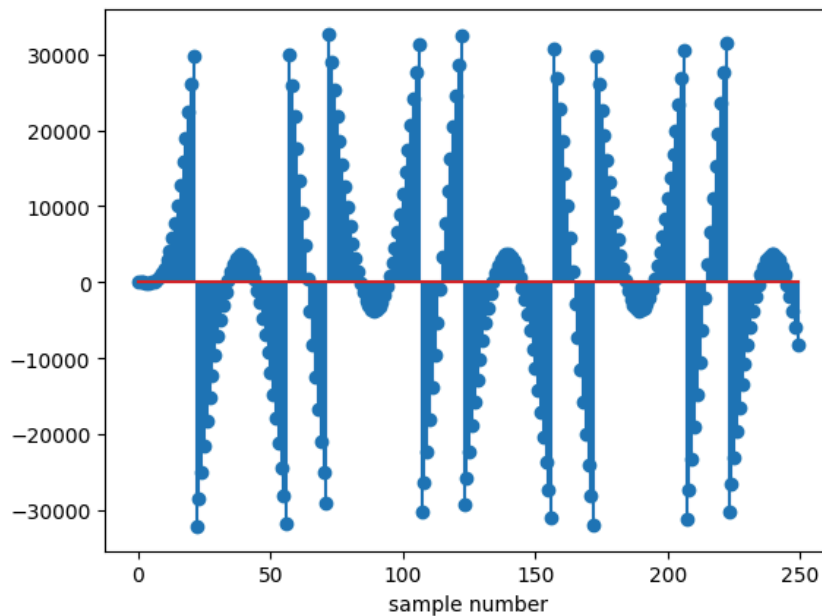
Our sample period is $1/44100 = 2.27\text{e-}5\text{s}$ or $22.7\mu\text{s}$. Therefore our filter speed is slower than our sample rate and thus would not be ‘good enough’ for real time filtering. Since we are only barely slower than the sample rate we would likely be able to easily improve/optimize the code for our filter to be “good enough”, but this will not be done as its not requested and we expect additional optimization methods to be covered in future lectures/labs.

Q2>> Screenshots of the first 250 filtered samples and the spectrum (from fixed point FIR)

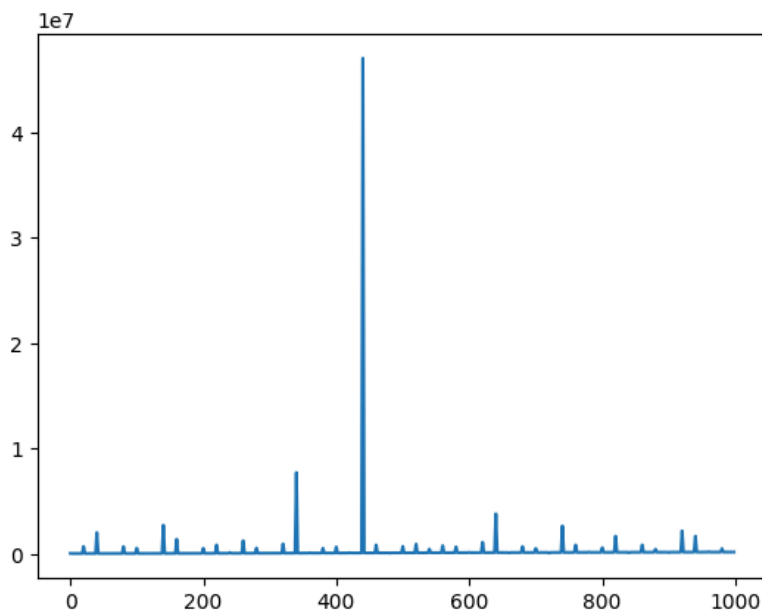
memory address of data

```
newdata: uint16_t [2... 0x20000244 <newdata>  
> [0...! uint16_t [1... 0x20000244 <newdata>  
> [100 uint16_t [1... 0x20005064 <newdata+200...  
> [200 uint16_t [2... 0x20009e84 <newdata+400...  
+ Add new
```

Filtered data plotted. Clearly we are seeing overflow from the 16bit values due to the 2x gain added, causing the signal to regularly invert close to its peaks.



FFT of filtered signal. We still see our expected 440Hz element, but also some other frequency elements from the signal inverting due to overflow.



How much time is taken to filter the signal? Is this filtering going to be “good enough” for our sample rate?

Image of debug timing

Index	Type	Data	Cycles	Time(s)	Extra info
0	Sync	96	1987511 ?	?	No timestamp received ...
1	Sync	48	1987511 ?	?	No timestamp received ...
2	ITM Port 31	1	1987511	20.703240 ms	
3	Sync	48	11987506	124.869854 ms	
4	Sync	48	27987498	291.536438 ms	
5	Sync	48	45987489	479.036344 ms	
6	ITM Port 31	2	51592776	537.424750 ms	
7	Sync	48	0	0.000000 s	

we can see that the filter took $[537.42\text{ms} - 20.70\text{ms}] = 516.72\text{ms}$ to filter the 22050 samples. This equates to $2.34\text{e-}5\text{s}$ per sample or $23.4\mu\text{s}$.

If we wanted to filter in real time, then the speed of our filtering must be at least the speed of our sampling, but should be faster in case the system needs to execute interrupts or other asynchronous tasks.

Our sample period is $1/44100 = 2.27\text{e-}5\text{s}$ or $22.7\mu\text{s}$. Therefore our filter speed is slower than our sample rate and thus would not be ‘good enough’ for real time filtering. Since we are only barely slower than the sample rate we would likely be able to easily improve/optimize the code for our filter to be “good enough”, but this will not be done as its not requested and we expect additional optimization methods to be covered in future lectures/labs.

PART TWO

Q1: How many seconds of audio is there in the input data?

The data in noisy.bin has $0\text{x}3\text{E}82\text{C}$ bytes of data, or 256,044 in base 10. Removing the header information, we have 256,000 bytes. Since we have duplicate data for left and right channels that would be played in parallel, we actually have $256,000/2$ bytes or 128,000 bytes. Since our data is actually 16 bits or 2 bytes we have $128,000/2$ or 64000 samples. When sampled at 8KHz we have $64000/8000$ seconds or 8 seconds of audio.

Q2> Run the debugger until filteredOutBufferA is full. Take a screenshot of the contents of the buffer, as below. Do you get the expected output?

Since there hasn’t been any discussion on what the ‘expected output’ should be, what the filter is doing or what the ‘noisy.bin’ data contains, its hard to be certain, but the data appears sinusoidal in nature which is indicative of an audio sample, and the first value matches what is shown in the lab handout, so i would say to the best of my knowledge that the output is as expected. Output seen below.

filteredOutBufferA	volatile int32_t [32]	0x200005dc <filteredOutBufferA>
filteredOutBufferA[0]	volatile int32_t	19398952
filteredOutBufferA[1]	volatile int32_t	134088702
filteredOutBufferA[2]	volatile int32_t	231476684
filteredOutBufferA[3]	volatile int32_t	299241942
filteredOutBufferA[4]	volatile int32_t	330240943
filteredOutBufferA[5]	volatile int32_t	320213782
filteredOutBufferA[6]	volatile int32_t	270995495
filteredOutBufferA[7]	volatile int32_t	187173672
filteredOutBufferA[8]	volatile int32_t	79430844
filteredOutBufferA[9]	volatile int32_t	-39846496
filteredOutBufferA[10]	volatile int32_t	-195388227
filteredOutBufferA[11]	volatile int32_t	-253693727
filteredOutBufferA[12]	volatile int32_t	-321786670
filteredOutBufferA[13]	volatile int32_t	-351147246
filteredOutBufferA[14]	volatile int32_t	-336729106
filteredOutBufferA[15]	volatile int32_t	-280563897
filteredOutBufferA[16]	volatile int32_t	-188418875
filteredOutBufferA[17]	volatile int32_t	-71697478
filteredOutBufferA[18]	volatile int32_t	55771987
filteredOutBufferA[19]	volatile int32_t	177343122
filteredOutBufferA[20]	volatile int32_t	277811343
filteredOutBufferA[21]	volatile int32_t	343807102
filteredOutBufferA[22]	volatile int32_t	366876126
filteredOutBufferA[23]	volatile int32_t	342365288
filteredOutBufferA[24]	volatile int32_t	272633920
filteredOutBufferA[25]	volatile int32_t	164956629
filteredOutBufferA[26]	volatile int32_t	32047593
filteredOutBufferA[27]	volatile int32_t	-111675048
filteredOutBufferA[28]	volatile int32_t	-249499359
filteredOutBufferA[29]	volatile int32_t	-366220756
filteredOutBufferA[30]	volatile int32_t	-448338617
filteredOutBufferA[31]	volatile int32_t	-488185113
k	int	32

Q3> Run the debugger again until filteredOutBufferA is full. Take a screenshot of the contents of the buffer, as below. Do you get the expected output?

Similar to Q2, I feel it difficult to provide a confident answer without more background, but since we do not get the same data as in Q2, I would say the data is not the expected outcome.

▼ filteredOutBufferA	volatile int32_t [32]	0x200005dc <filteredOutBufferA>
filteredOutBufferA[0]	volatile int32_t	20775229
filteredOutBufferA[1]	volatile int32_t	-22217043
filteredOutBufferA[2]	volatile int32_t	-56361820
filteredOutBufferA[3]	volatile int32_t	-80217288
filteredOutBufferA[4]	volatile int32_t	-92145022
filteredOutBufferA[5]	volatile int32_t	-93390225
filteredOutBufferA[6]	volatile int32_t	-84935952
filteredOutBufferA[7]	volatile int32_t	-70255664
filteredOutBufferA[8]	volatile int32_t	-51905304
filteredOutBufferA[9]	volatile int32_t	-33358333
filteredOutBufferA[10]	volatile int32_t	-17563916
filteredOutBufferA[11]	volatile int32_t	-6357089
filteredOutBufferA[12]	volatile int32_t	-917518
filteredOutBufferA[13]	volatile int32_t	-1114129
filteredOutBufferA[14]	volatile int32_t	-5767256
filteredOutBufferA[15]	volatile int32_t	-12910789
filteredOutBufferA[16]	volatile int32_t	-20119859
filteredOutBufferA[17]	volatile int32_t	-24969597
filteredOutBufferA[18]	volatile int32_t	-25297282
filteredOutBufferA[19]	volatile int32_t	-20054322
filteredOutBufferA[20]	volatile int32_t	-8847495
filteredOutBufferA[21]	volatile int32_t	7077996
filteredOutBufferA[22]	volatile int32_t	25756041
filteredOutBufferA[23]	volatile int32_t	44303012
filteredOutBufferA[24]	volatile int32_t	59507596
filteredOutBufferA[25]	volatile int32_t	67896332
filteredOutBufferA[26]	volatile int32_t	67044351
filteredOutBufferA[27]	volatile int32_t	55116617
filteredOutBufferA[28]	volatile int32_t	31785445
filteredOutBufferA[29]	volatile int32_t	-2293795
filteredOutBufferA[30]	volatile int32_t	-44761771
filteredOutBufferA[31]	volatile int32_t	-92407170
k	int	32

Q4> How much time/how many cycles are required for each sample, with the original ProcessSample function? Take a screen shot of the generated assembly code and explain where you might think the main bottlenecks are.

As seen in the images below, it takes [61632 - 48684] cycles or 12,948 cycles to complete a call to ProcessSample. This is also 129us.

```

174     ITM_Port32(0) = 1;
175     filteredSampleL = ProcessSample(newSampleL,history_1); // "L"
176     ITM_Port32(0) = 2;

```

Index	Type	Data	Cycles	Time(s)	Extra info
0	Sync	56	48237 ?	?	No timestamp received ...
1	ITM Port 30	0	48237	482.370000 µs	
2	ITM Port 0	1	48684	486.840000 µs	
3	ITM Port 30	10	61355	613.550000 µs	
4	ITM Port 0	2	61632	616.320000 µs	
5	Sync	48	0	0.000000 s	

The assembly code starts here.

```

174          ITM_Port32(0) = 1;
08000f7c:   mov.w   r3, #3758096384 @ 0xe0000000
08000f80:   movs    r2, #1
08000f82:   str     r2, [r3, #0]
175          filteredSampleL = ProcessSample(newSampleL,history_1); // "L"
08000f84:   ldr     r3, [pc, #188] @ (0x8001044 <main+332>)
08000f86:   ldrsh.w r3, [r3]
08000f8a:   ldr     r1, [pc, #188] @ (0x8001048 <main+336>)
08000f8c:   mov     r0, r3
08000f8e:   bl      0x800127c <ProcessSample>
08000f92:   mov     r3, r0
08000f94:   mov     r2, r3
08000f96:   ldr     r3, [pc, #180] @ (0x800104c <main+340>)
08000f98:   strh    r2, [r3, #0]
*176          ITM_Port32(0) = 11;

```

The ProcessSample assembly code is below.

```

365      static int16_t ProcessSample(int16_t newsample, int16_t* history) {
      ProcessSample:
0800127c:   push    {r7}
0800127e:   sub     sp, #28
08001280:   add     r7, sp, #0
08001282:   mov     r3, r0
08001284:   str     r1, [r7, #0]
08001286:   strh    r3, [r7, #6]
368          history[0] = newsample;
08001288:   ldr     r3, [r7, #0]
0800128a:   ldrh    r2, [r7, #6]
0800128c:   strh    r2, [r3, #0]
371          int tap = 0;
0800128e:   movs    r3, #0
08001290:   str     r3, [r7, #20]
372          int32_t accumulator = 0;
08001292:   movs    r3, #0
08001294:   str     r3, [r7, #16]
373          for (tap = 0; tap < NUMBER_OF_TAPS; tap++) {
08001296:   movs    r3, #0
08001298:   str     r3, [r7, #20]
0800129a:   b.n     0x80012c2 <ProcessSample+70>
374          accumulator += (int32_t)filter_coeffs[tap] * (int32_t)history[tap];
0800129c:   ldr     r2, [pc, #188] @ (0x800135c <ProcessSample+224>)
0800129e:   ldr     r3, [r7, #20]
080012a0:   ldrsh.w r3, [r2, r3, lsl #1]
080012a4:   mov     r1, r3
080012a6:   ldr     r3, [r7, #20]
080012a8:   lsls    r3, r3, #1
080012aa:   ldr     r2, [r7, #0]
080012ac:   add     r3, r2
080012ae:   ldrsh.w r3, [r3]
080012b0:   mul.w   r3, r1, r3
080012b2:   ldr     r2, [r7, #16]
080012b4:   add     r3, r2
080012b6:   str     r3, [r7, #16]
375          for (tap = 0; tap < NUMBER_OF_TAPS; tap++) {
080012b8:   ldr     r3, [r7, #20]
080012ba:   adds    r3, #1
080012bc:   str     r3, [r7, #20]
080012be:   ldr     r3, [r7, #20]
080012c0:   cmp     r3, #219 @ 0xdb
080012c2:   ble.n   0x800129c <ProcessSample+32>
377          ITM_Port32(0) = 2;
080012c4:   mov.w   r3, #3758096384 @ 0xe0000000
080012c6:   movs    r2, #2
080012c8:   str     r2, [r3, #0]
380          for(tap = NUMBER_OF_TAPS-2; tap > -1; tap--) {
080012ca:   movs    r3, #218 @ 0xda
080012cc:   str     r3, [r7, #20]
080012ce:   b.n     0x80012f4 <ProcessSample+120>
381          history[tap+1] = history[tap];

```

```

381          history[tap+1] = history[tap];
080012d6:   ldr     r3, [r7, #20]
080012d8:   lsls    r3, r3, #1
080012da:   ldr     r2, [r7, #0]
080012dc:   add     r2, r3
080012de:   ldr     r3, [r7, #20]
080012e0:   adds    r3, #1
080012e2:   lsls    r3, r3, #1
080012e4:   ldr     r1, [r7, #0]
080012e6:   add     r3, r1
080012e8:   ldrsh.w r2, [r2]
080012ec:   strh    r2, [r3, #0]
380          for(tap = NUMBER_OF_TAPS-2; tap > -1; tap--) {
080012ee:   ldr     r3, [r7, #20]
080012f0:   subs    r3, #1
080012f2:   str     r3, [r7, #20]
080012f4:   ldr     r3, [r7, #20]
080012f6:   cmp     r3, #0
080012f8:   bge.n   0x80012d6 <ProcessSample+90>
384          ITM_Port32(0) = 3;
080012fa:   mov.w   r3, #3758096384 @ 0xe0000000
080012fe:   movs    r2, #3
08001300:   str     r2, [r3, #0]
386          if (accumulator > 0x3FFFFFFF) {
08001302:   ldr     r3, [r7, #16]
08001304:   cmp.w   r3, #1073741824 @ 0x40000000
08001306:   blt.n   0x800131c <ProcessSample+160>
387          accumulator = 0x3FFFFFFF;
0800130a:   mvn.w   r3, #3221225472 @ 0xc0000000
0800130e:   str     r3, [r7, #16]
388          overflow_count++;
08001310:   ldr     r3, [pc, #76] @ (0x8001360 <ProcessSample+228>)
08001312:   ldr     r3, [r3, #0]
08001314:   adds    r3, #1
08001316:   ldr     r2, [pc, #72] @ (0x8001360 <ProcessSample+228>)
08001318:   str     r3, [r2, #0]
0800131a:   b.n     0x8001334 <ProcessSample+184>
389          } else if (accumulator < -0x40000000) {
0800131c:   ldr     r3, [r7, #16]
0800131e:   cmp.w   r3, #3221225472 @ 0xc0000000
08001320:   bge.n   0x8001334 <ProcessSample+184>
390          accumulator = -0x40000000;
08001322:   mov.w   r3, #3221225472 @ 0xc0000000
08001324:   str     r3, [r7, #16]
391          underflow_count++;
08001326:   ldr     r3, [pc, #56] @ (0x8001364 <ProcessSample+232>)
08001328:   ldr     r3, [r3, #0]
0800132a:   adds    r3, #1
0800132c:   ldr     r2, [pc, #48] @ (0x8001364 <ProcessSample+232>)
0800132e:   str     r3, [r2, #0]
394          ITM_Port32(0) = 4;
08001330:   mov.w   r3, #3758096384 @ 0xe0000000
08001332:   movs    r2, #4
08001334:   str     r2, [r3, #0]
396          int16_t temp = (int16_t)(accumulator >> 15);
08001336:   ldr     r3, [r7, #16]
08001338:   asrs    r3, r3, #15
0800133a:   strh    r3, [r7, #14]
398          ITM_Port32(0) = 5;
0800133c:   mov.w   r3, #3758096384 @ 0xe0000000
0800133e:   movs    r2, #5
08001340:   str     r2, [r3, #0]
399          return temp;

```


Index	Type	Data	Cycles	Time(s)
0	Sync	56	39034 ?	?
1	ITM Port 30	0	39034	390.340000 μ s
2	ITM Port 0	1	39481	394.810000 μ s
3	ITM Port 0	2	46572	465.720000 μ s
4	ITM Port 30	10	52151	521.510000 μ s
5	ITM Port 0	3	52397	523.970000 μ s
6	Sync	48	52438 ?	?
7	ITM Port 0	4	52438	524.380000 μ s

Through initial probing with ITM calls, it appears that the chunks of C code that take the longest are the convolution operation from lines 368 to 377 taking around 7000 cycles, and the shuffling operation on the history variable from lines 377 to 384 taking around 6000 cycles. Looking at the assembly code we can also see that the operation that takes the most assembly operations is convolution.

We could dig in further to the exact lines that are problematic, but ultimately we would need to spend effort optimizing the convolution operation and or the shuffling operation to reduce the bottle neck.

Q5> Create a new ProcessSample2 function, making use of the appropriate MAC instruction. You might remember from class that we used in-line assembly for this. How much time is required for each sample? Do we satisfy timing requirements now? Screenshot/copy your function and explain the changes that you made.

Below is an implementation of the multiple and sum MAC instruction as seen in our lecture.

Below is an implementation of the multiply and sum MAC instruction as seen in our lecture

Additionally, you can see that the ProcessSample function now takes 11,957 cycles and 119.57us. This may appear as a meager improvement over the previous function, yet our signal processing is now faster than our interrupt call happening every 125us and thus meet timing requirements.

```
ITM_Port32(0) = 1;
// filteredSample1 = ProcessSample(newSample1,history_1); // "L"
filteredSample1 = ProcessSample2(newSample1,history_1); // "L"
// filteredSample1 = ProcessSample3(newSample1,history_1); // "L"
ITM_Port32(0) = 2;
```

Below you can see the original line of code commented out that handled the multiply and add operation replaced with the three lines of code below it where we now utilize a MAC SMLABB function for convolution.

```
for (tap = 0; tap < NUMBER_OF_TAPS; tap++) {
// accumulator += (int32_t)filter_coeffs[tap] * (int32_t)history[tap];

__asm volatile ("SMLABB %[result], %[op1], %[op2], %[acc]"
: [result] "=r" (accumulator)
: [op1] "r" (history[tap]), [op2] "r" (filter_coeffs[tap]), [acc] "r" (accumulator)
);
}
```

Index	Type	Data	Cycles	Time(s)	Extra info
0	Sync	56	42172 ?	?	No timestamp received ...
1	ITM Port 30	0	42172	421.720000 μ s	
2	ITM Port 0	1	42628	426.280000 μ s	
3	ITM Port 0	2	54585	545.850000 μ s	
4	Sync	48	0	0.000000 s	

Q6> Create a new ProcessSample3 function, making use of the appropriate SIMD instruction. You might remember from class that we might be able to use an intrinsic for this. How much time is required for each

sample? Do we satisfy timing requirements now? Screenshot/copy your function and explain the changes that you made.

Each sample or call to ProcessSample3 takes 8876 cycles and 88.76us. A substantial improvement utilizing parallel operations. This improvement should certainly allow us to meet timing requirements since its now much faster than the 125us interrupt call.

```
#endif
    ITM_Port32(0) = 1;
    // filteredSampleL = ProcessSample(newSampleL,history_1); // "L"
    // filteredSampleL = ProcessSample2(newSampleL,history_1); // "L"
    filteredSampleL = ProcessSample3(newSampleL,history_1); // "L"
    ITM_Port32(0) = 2;
```

Below you can see the code implementing multiply and accumulate using the SIMD SMLAD function. A notable change is the incrementation of the for loop. Since we are now performing 2 operations at once, we need to increment by 2 to get the next set of samples.

```
for (tap = 0; tap < NUMBER_OF_TAPS; tap+=2) {
    // accumulator += (int32_t)filter_coeffs[tap] * (int32_t)history[tap];

    __asm volatile ("SMLAD %[result], %[op1], %[op2], %[acc]"
    : [result] "+r" (accumulator)
    : [op1] "r" (history[tap]), [op2] "r" (filter_coeffs[tap]), [acc] "r" (accumulator)
    );
}
```

Index	Type	Data	Cycles	Time(s)
0	Sync	56	39881 ?	?
1	ITM Port 30	0	39881	398.810000 µs
2	ITM Port 0	1	40337	403.370000 µs
3	ITM Port 0	2	49213	492.130000 µs
4	Sync	48	0	0.000000 s

Q7> Write a reflection of what you have observed and learned in this lab, including an explanation of how you checked that your code optimizations preserved functional correctness.

Experimenting with the the fix point and floating point filtering was interesting to see how important it is to know what your filter is doing to your data and pay attention to situations where you might see over flow as we did. Additionally, it was interesting to see the impact of data types on performance and processor demand.

Finally, it was a great learning experience to test C code for convolution against the two optimized functions for multiply and accumulate. It makes me wonder what else can be as easily implemented to improve processing speed for future projects. Certainly something I will look out for!

To validate that the changes I made through steps 5, 6, 7, I would uncomment **FUNCTIONAL_TET_1** and fill bufferA to ensure I would continue to get the same result. I regularly used this and the ITM outputs on SWV log for debugging.